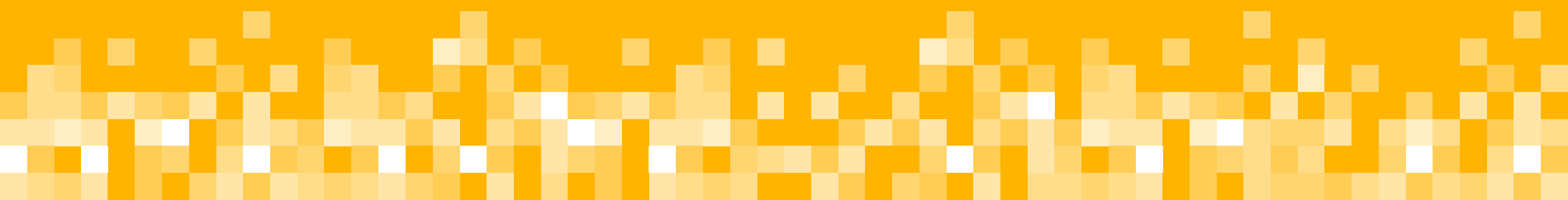


Moduł 3

Refaktoryzacja do wzorca Aggregate

Ćwiczenia praktyczne



Zadanie:

Refaktoryzacja braku spójności

Wykonaj komendę:

```
git checkout podatki-rozwiazanie
```

w odpowiednim projekcie:

- ➔ <https://github.com/legacyfighter/dodatkowe-zadania-java>
- ➔ <https://github.com/legacyfighter/dodatkowe-zadania-php>
- ➔ <https://github.com/legacyfighter/dodatkowe-zadania-csharp>

i zobacz na zbiór klas związanych z podatkami zamówień dietetycznych -- legacyfighter.dietary.TaxRule, TaxConfig, TaxRuleService, etc. W telegraficznym skrócie - dla każdego kraju (identyfikacja przez kod kraju) przypisana jest konfiguracja TaxConfig. Konfiguracja zawiera wiele zasad aplikowanych w danym kraju (zbiór TaxRule). Dokonując inżynierii wstecznej, **znajdź reguły biznesowe**, które zawsze muszą być spójne (przynajmniej tak wynika z kodu). Wypisz je i zlokalizuj miejsce w kodzie, które za nie odpowiada. Przykładem podanym przez biznes jest następująca reguła:

- 1) "Krajowa konfiguracja podatkowa musi mieć zawsze przynajmniej jedną regułę podatkową" - reprezentowana przez następujący kawałek kodu przy usuwaniu reguły:

```
if (taxConfig.getTaxRules().size() == 1) {  
    throw new IllegalStateException("Last rule in country config");  
}
```

Problemy, na które od wielu miesięcy narzeka interesariusz biznesowy związany z podatkami:

- 1) niespójność danych. Zdarza się, że w systemie łamane są reguły (mamy więcej lub mniej reguł podatkowych w kraju niż powinno to być możliwe)
- 2) W systemie brakuje często wpisów na temat ostatniej modyfikacji konfiguracji krajowej
- 3) Dla jednego specyficznego kraju, ładowanie zasad podatkowych i ich dodawanie trwa bardzo długo lub kończy się wyjątkiem (problem o mniejszym priorytecie).

Zaproponuj refaktoryzację/naprawę obecnej sytuacji. Załóż, że zespół programistyczny ma dowolność w wywoływaniu setterów z różnych miejsc systemu oraz robi to często.

Napisz kilka testów na reguły biznesowe związane z podatkami. Pamiętaj, aby:

- 1) Znaleźć odpowiedni poziom "wysokości testów" - taki, który pozwoli na swobodę testowania
- 2) Znaleźć obserwowalne zachowania (komendy i query), które znajdują się w funkcjach biznesowych związanych z podatkami.

Po napisaniu testów **zaproponuj i zaimplementuj obiekt stanowiący granicę spójności odpowiednich reguł biznesowych**. Pamiętaj o:

- 1) Bezpiecznej refaktoryzacji kodu serwisowego w TaxRuleService

- 2) Upewnieniu się czy inne funkcje biznesowe w systemie nie korzystają z danych agregatu - jeśli tak, jak zapewnić enkapsulację i spójność?
Podpowiedź: Zastanów się DO CZEGO inne części systemu wykorzystują dane

W pierwszej kolejności zajmij się po prostu API i implementacją obiektu. Nie przejmuj się jeśli obiekt jest zbyt duży albo jeśli nieczytelny. **Pamiętaj, jaki problem rozwiązujesz.** Przykładowe (poglądowe, nie wzorcowe) rozwiązanie tego pierwszego kroku znajdziesz wykonując komendę:

```
git checkout podatki-rozwiazanie
```

W kolejnych krokach możesz zastanowić się nad poprawą granic tego obiektu albo nad zwiększeniem czytelności. Jak? Na przykład destylując w nim obiekty wartości (Value Objects).